BÖLÜM 1: BÜYÜK UYANIŞ — MIMAR OLARAK İLK GÜN

⏱ 8 dk • DERS 1

Parça 1: Kod Yazmanın Yetmediği Gün: Bir Yazılım Mimarı Aslında Ne Yapar?

Pazartesi sabahı sprint panosunda sana atanmış tek bir Jira görevi bile olmadığını fark ettiğin an.

🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**

Mert, şirketin efsanevi 10x kıdemli yazılımcısıydı. Öğle yemeğine çıkmadan önce 4.000 satırlık spagetti TypeScript kodunu tek başına refactor edebilir, uykusunda veritabanı sorgularını optimize edebilirdi. Cuma günü akşamüzeri Mühendislikten Sorumlu Başkan Yardımcısı sırtını sıvazladı: 'Mert, artık yeni Yazılım Mimarımız sensin!' Pazartesi sabahı 09:00'da Mert heyecanla masasına oturdu, IDE'sini açtı ve sprint panosuna baktı. Kendisine atanmış tek bir bilet bile yoktu. Takım liderine gidip 'Bana ne iş var?' diye sordu. Aldığı yanıt donup kalmasına yetti: 'Biz ne inşa edeceğimizi sana soracaktık.' Mert, kendini faydalı hissetmek için öğleye kadar açık kaynaklı bir kütüphanenin girintilerini düzeltti. Saat 14:00 olduğunda acı gerçeği anladı: Artık kimse ona bir yol haritası vermeyecekti. Yol haritasını çizen kişi artık bizzat kendisiydi.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Bir geliştiriciyken değerin somut çıktılarla ölçülürdü: birleştirilen pull request'ler, çözülen bug'lar, yazılan satırlar. Bir mimar olarak ise para birimin karar kalitesi, sistem sınırları ve organizasyonel uyumdur. Ralph Johnson'ın meşhur tanımıyla mimarlık: 'Gelecekte değiştirilmesi zor olan tüm kararlardır.' Eğer üretkenliğini IDE'de yazdığın kod satırıyla ölçmeye devam edersen, mimarlık rolünde kendini sürekli işsiz ve yetersiz hissedersin. Senin işin artık roket motorunu tek başına inşa etmek değil; bir bisiklete roket motoru takmaya kalkışmadığımızdan emin olmaktır.

Temel Mimari İlkeler ve Zihinsel Modeller

Yazılım Mimarisinin Üç Temel Direği

1. **Yapı ve Ayrıştırma (Decomposition):** Karmaşık ve anlaşılmaz devasa bir iş probleminin, net sınırlarla yönetilebilir küçük modüllere bölünmesi.
2. **Sistem Nitelikleri (The '-ilities'):** Ölçeklenebilirlik, erişilebilirlik, bakım kolaylığı, güvenlik ve test edilebilirlik. Bir sistem işlevsel olarak kusursuz çalışsa bile yük altında 14 saniyede yanıt veriyorsa mimari açıdan ölüdür.
3. **Ödünleşme Yönetimi (Trade-offs):** Her mimari seçimin içinde hem avantajları hem de gizli bakım vergilerini barındıran bir paket anlaşma olduğunu kabul etmek.

Mimarın Zaman Ufku

Geliştiriciler mevcut sprint içinde yaşar (1-2 hafta). Takım liderleri çeyreklik hedeflere bakar (3 ay). Yazılım mimarı ise 1 ila 3 yıllık ufukta yaşamak zorundadır. Şirket başarılı olduğunda trafiğin, veri hacminin ve ekip büyüklüğünün nereye varacağını bugünden öngörmeli, ancak bugünden gereksiz spekülatif aşırı-mühendislik tuzağına düşmemelidir.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Fildişi Kule Astronotu:** Kod tabanından tamamen kopup kimsenin okumadığı ve gerçekte uygulanamayacak 80 sayfalık Visio diyagramları çizen mimar.
- **Gizli Kod Korsanı:** Toplantılardan bunalıp gece 23:00'te sisteme gizlice girerek junior geliştiricilerin kodlarını 'anlatmakla vakit kaybetmeyeyim' diyerek yeniden yazan mimar.
- **Hacker News Turisti:** Hacker News'te gördüğü her yeni ve deneysel teknolojiyi prodüksiyona sokup şirketi alfa sürüm kobayına çeviren mimar.

🎯 MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR

- **1. Adım:** Sistemin mevcut 'Değiştirilmesi En Zor' kararlarının envanterini çıkar: Veritabanı, ana mesajlaşma protokolü ve monolit sınırları.
- **2. Adım:** Ekipten en çok şikayet eden 3 kıdemli yazılımcıyla 30'ar dakikalık birebir görüş: 'Mevcut mimari her gün hangi noktada canınızı sıkıyor?'
- **3. Adım:** Temel başarım metriğini belirle: Mimari akış hızı (Yeni bir geliştirici, sistemi çökertme korkusu yaşamadan ne kadar hızla güvenli kod canlıya alabiliyor?).

Mimari Çıkarımlar ve Hile Tablosu

- Mimarlık kutu çizmek değil; gelecekte hangi dertlerden muzdarip olmak istediğini bugünden seçmektir.
- Kod, yazılımcı için bir varlık (asset); mimar için ise bir yükümlülüktür (liability). Yazılan her satır kod güncellenmek, yamalanmak ve bakımı yapılmak zorundadır.
- En iyi mimarlar kodu başkalarına iş bırakmamak için değil, problemi derinlemesine hissetmek için yazar.

🎙️ UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU

Ders Değerlendirmesi ve Ödev: Şu an çalıştığın projeyi düşün: 2 yıl önce alınan ve bugün tüm ekibin şikayet ettiği tek bir mimari karar söyle. O gün o kararın mantıklı görünmesini sağlayan şartlar neydi?